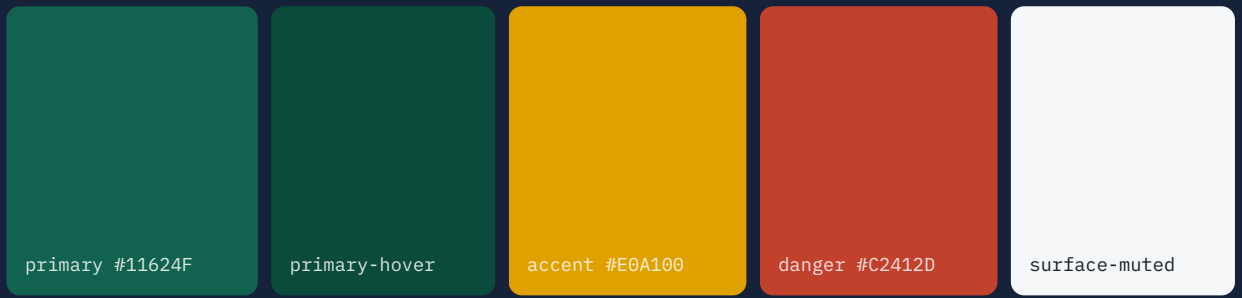
WORKED EXAMPLE · BUILD REPORT WITH PROOF

Building Ledgerly DS, step by step

One design system package, built from scratch and installed into real Angular, React, Bootstrap, Tailwind, Material UI and plain HTML apps. Every step documented, every claim tested.



Contents · At a glance · Steps 1–10 in detail · Pilot findings & fixes · Brand-change proof · Verification summary · Handover

OVERVIEW

What was built, at a glance

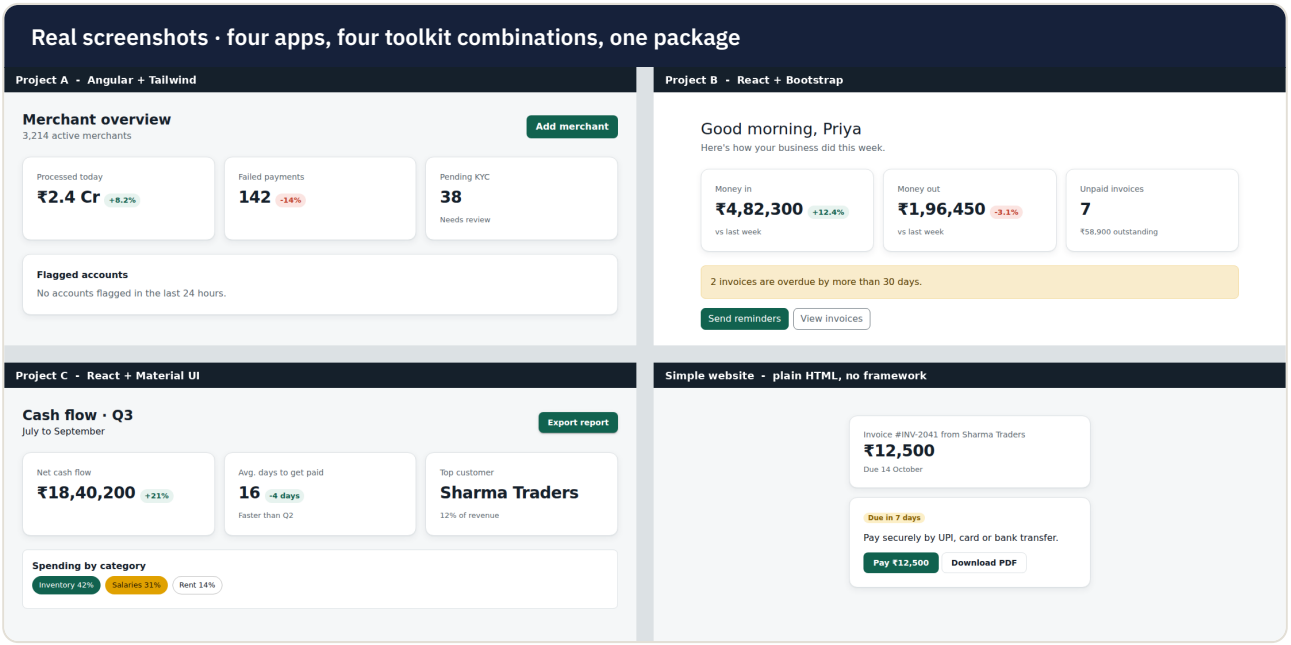
Ledgerly, a fictional small-business finance company, has your exact situation: several apps, each on a different toolkit combination, and one design system that must serve them all.

25	5	1	4	0
design tokens (11 raw, 14 meanings)	auto-generated adapters	package, 13 compartments	real apps built and screenshotted	console errors in any app

The apps

APP	PURPOSE	BUILT WITH	STYLED WITH	STATUS
Project A · Ledgerly Admin	Staff portal	Angular 20	Tailwind 4	**Built & running**
Project B · Ledgerly Web	Customer website	React 19	Bootstrap 5.3	**Built & running**
Project C · Ledgerly Insights	Analytics dashboard	React 19	Material UI	**Built & running**
Simple site · Pay an invoice	Public payment page	None	Plain CSS	**Running**
Future · Vue app	Template for later	Vue 3	Vuetify 3	Code provided

The result



Captured from the actual built apps running in a browser. Same brand, corners, spacing and money card, all from one master list.

Step 1

Take stock of what you have

Owner: UX architect

Before building anything, list the design choices, the components, and every toolkit combination that has to be supported.

Design choices gathered

CATEGORY	WHAT LEDGERLY USES
Colors	A deep "ledger green" brand, a saffron accent, red for danger, blue for information, and five slate neutrals from white to near-black
Typography	Manrope, with system fallbacks; four sizes (14, 16, 20, 28 px) and two weights
Spacing	Five steps: 4, 8, 16, 24, 32 px
Corners	Small 4 px, medium 8 px, large 14 px
Depth	One soft card shadow

Component inventory

COMPONENT	DECISION	REASON
Button, input, alert, chip, card	Kits already have these	Bootstrap, Material UI and Tailwind all provide them. They only need repainting with Ledgerly tokens.
Money summary card	**Unique to Ledgerly**	Every Ledgerly app shows totals with a trend ("₹4,82,300, +12.4%"). No kit has this, so it's built once for everyone.

Why this step matters

It keeps the build small. Instead of rebuilding dozens of components for every framework, only one genuinely unique component needs building; everything else comes from the kits each team already uses.

Step 2

Turn choices into named tokens

Owner: UX architect (with designer)

Every choice gets a name, in two levels: raw paint, and what each paint is for.

Level 1 · Raw values (primitives): the paint card, never used directly by apps



Level 2 · Meanings (semantic tokens)

What apps actually use. Each meaning points at a raw value, so rebranding means changing a pointer, not hunting through code.

MEANING	POINTS TO	USED FOR
`color.primary`	green-600	Main buttons, links, focus rings
`color.primary-subtle`	green-50	Positive trend chips, soft highlights
`color.accent`	saffron-500	Secondary highlights, warnings
`color.danger`	red-600	Errors, negative trends
`color.surface` / `surface-muted`	slate-0 / slate-50	Cards / page background

...plus primary-hover, on-primary, success, warning, info, border, text and text-muted: 14 meanings in total.

How it looks in the file (W3C token format, as exported by Figma)

```
// tokens/semantic.json (excerpt)
{
  "color": {
    "primary": { "$type": "color", "$value": "{palette.green.600}",
                 "$description": "Main action color: buttons, links, focus" },
    "danger":  { "$type": "color", "$value": "{palette.red.600}" }
  }
}
```

Step 3

Create the box

Owner: developer

One project folder that becomes the package. Tools used: Node.js 22, npm, Style Dictionary 4.4.

```
ledgerly-ds/
├── tokens/
│   ├── primitives.json        ← raw paint (Step 2)
│   └── semantic.json          ← meanings (Step 2)
├── build/
│   ├── build-tokens.mjs       ← the translator (Step 4)
│   └── copy-src.mjs
├── src/
│   ├── styles/styles.css      ← plain classes for kit-free pages
│   ├── components/            ← <ds-stat-card> web component (Step 5)
│   ├── react/  angular/  vue/ ← thin wrappers (Step 5)
├── dist/                      ← GENERATED output, never edited by hand
├── examples/                  ← four consuming apps (Step 9)
├── package.json               ← compartment labels (Step 6)
└── README.md
```

The one rule of this folder

Humans edit `tokens/` and `src/`. Machines write `dist/`. Every generated file starts with a warning saying "Do not edit — change tokens/*.json instead."

Step 4

Set up the automatic translator

Owner: developer

One script reads the master list and writes a version for every kit. One command, `npm run build`, regenerates all of them.

OUTPUT FILE	WHAT IT CONTAINS	WHY THIS FORM
`tokens.css`	CSS variables, with meanings pointing at raw values	Universal; used by everything
`tailwind.css`	Tailwind v4 theme mapping its names to our variables	Tailwind classes follow the tokens live
`bootstrap.scss`	Bootstrap Sass variables with real hex values	Bootstrap does colour maths on them
`mui/theme.js`	A Material UI theme object	MUI also calculates shades internally
`vuetify/theme.js`	A Vuetify theme object	Ready for future Vue apps
`tokens.json`	All tokens resolved, as data	For docs, tooling, other platforms

What the translator actually produced

Excerpts from the real generated files. Notice the same green, `#11624F`, flowing into every format.

```
/* dist/tokens.css: meanings point at raw paint */
--ds-palette-green-600: #11624f;
--ds-color-primary: var(--ds-palette-green-600);  /** Main action color */
--ds-color-text-muted: var(--ds-palette-slate-500);
```

```
/* dist/tailwind.css: Tailwind's names → our variables */
@theme inline {
  --color-primary: var(--ds-color-primary);
  --spacing-md:    var(--ds-space-md);
  --radius-lg:     var(--ds-radius-lg);
}
```

```scss
// dist/bootstrap.scss: real values for Bootstrap's Sass
$primary:       #11624F;
$danger:        #C2412D;
$border-radius: 8px;
$font-family-sans-serif: 'Manrope', system-ui, -apple-system, 'Segoe UI', sans-serif;
```

```js
// dist/mui/theme.js: Material UI's shape
export const ledgerlyMuiTheme = {
  palette: { primary: { main: "#11624F", dark: "#0B4B3C", contrastText: "#FFFFFF" }, … },
  shape: { borderRadius: 8 },
  typography: { fontFamily: "'Manrope', system-ui, …", button: { textTransform: "none" } }
};
```

✓ Proof · build log

```
$ npm run build
✔ dist/tailwind.css
✔ dist/bootstrap.scss
✔ dist/mui/theme.js
✔ dist/vuetify/theme.js
✔ dist/tokens.json
✔ dist/tokens.css
✔ copied src → dist
```

A design decision worth knowing

Tailwind and the plain layer use CSS variables, so they could even switch themes while the app runs. Bootstrap and Material UI receive real values, because both calculate hover and contrast shades themselves and can't do maths on a variable. The translator handles this difference automatically.

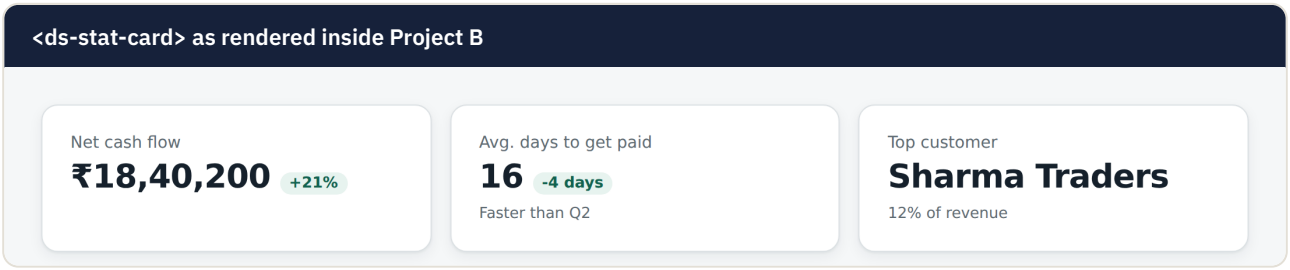
Step 5

Build the unique component once

Owner: developer, reviewed by UX architect

The money summary card, `<ds-stat-card>`, built as a native web component, which every browser understands with or without a framework.

<ds-stat-card> as rendered inside Project B



Label, value, trend chip (green or red) and caption. All colors, sizes and corners come from tokens.

Its public "contract"

INPUT	EXAMPLE	EFFECT
`label`	"Money in"	Small heading
`value`	"₹4,82,300"	The big number
`trend`	"+12.4%"	Chip; green if positive, red if it starts with "−"
`caption`	"vs last week"	Small note below
`tone` (added in pilot)	"positive"	Overrides the colour when a minus is good news
event `ds-select`	click, Enter, Space	Tells the app the card was chosen

Built-in quality

Keyboard accessible (focusable, Enter/Space activate), screen-reader label ("Money in: ₹4,82,300, +12.4%"), visible focus ring, styles sealed in Shadow DOM so no app's CSS can break it, and sensible fallbacks if token variables are missing.

The thin wrappers

Each framework gets a tiny adapter so the component feels native to that team. None of them contain design logic; they only translate.

FRAMEWORK	HOW THE TEAM WRITES IT	WHAT THE WRAPPER DOES
React	`<StatCard value="…" onSelect={fn} />`	Maps `onSelect` to the native event; works on React 18 and 19
Angular	`<ds-stat-card [value]="x" (dsSelect)="fn($event)">`	Typed inputs and output; compiled for Angular (see pilot)
Vue	`<ds-stat-card :value="x" @ds-select="fn" />`	Plugin tells Vue `ds-*` tags are web components
Plain HTML	`<ds-stat-card value="…">`	No wrapper needed at all

```
// src/react/index.js (core of the React wrapper)
export function StatCard({ label, value, trend, caption, tone, onSelect }) {
  const ref = useRef(null);
  useEffect(() => {
    const handler = (e) => onSelect?.(e.detail);
    ref.current.addEventListener('ds-select', handler);
    return () => ref.current?.removeEventListener('ds-select', handler);
  }, [onSelect]);
  return createElement('ds-stat-card', { ref, label, value, trend, caption, tone });
}
```

These wrappers were hand-written so the mechanics are visible. As the component library grows, Stencil can generate them automatically.

Step 6

Label the compartments

Owner: developer

One package, `@ledgerly/ds`, with a label for every door. Projects import only what they need.

```
// package.json (excerpt)
"exports": {
  "./tokens.css":     "./dist/tokens.css",
  "./tailwind.css":   "./dist/tailwind.css",
  "./bootstrap.scss": "./dist/bootstrap.scss",
  "./mui":            "./dist/mui/theme.js",
  "./vuetify":        "./dist/vuetify/theme.js",
  "./react":          "./dist/react/index.js",
  "./angular":        { "types": "./dist/angular/index.d.ts", "default": "./dist/angular/
index.js" },
  "./vue":            "./dist/vue/index.js"   …13 compartments in total
},
"peerDependenciesMeta": { "react": { "optional": true }, "@angular/core": { "optional":
true }, "vue": { "optional": true } }
```

> **Why "optional" matters**
>
> An Angular project installing `@ledgerly/ds` never downloads React, and a plain website downloads no framework at all.

Steps 7 & 8

Preview gallery and publishing

Being honest about scope: one of these was simulated, one is left for your team.

STEP	STATUS	DETAIL
7 · Storybook gallery	Not built here	Recommended next. The README lists it as a to-do. The four example apps currently serve as living documentation.
8 · Publish	Simulated exactly	`npm pack` produced `ledgerly-ds-1.1.0.tgz`, the identical file `npm publish` would upload. Every app installed from that file, not from the source folder, so the test reflects what real teams would receive.

✓ Proof · package created

```
$ npm pack
npm notice total files: 13
ledgerly-ds-1.1.0.tgz
```

To publish for real, your developer runs `npm publish` to a private npm organisation or GitHub Packages. After that, every project installs with the same single command: `npm install @ledgerly/ds`.

Step 9

The pilot: plugging into real apps

Owner: project developers

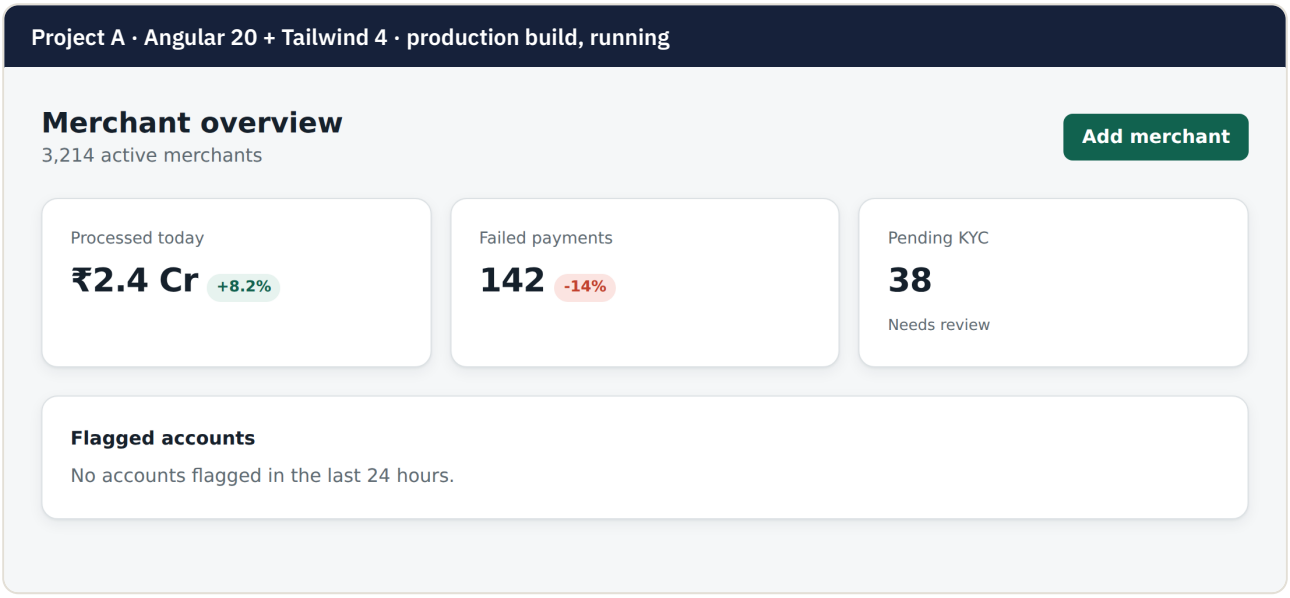
Each app was created, the package installed, the app built with its real toolchain, then opened in a browser and screenshotted.

Project A · Angular + Tailwind (Ledgerly Admin)

```
/* src/styles.css — the only setup needed for styling */
@import "tailwindcss";
@import "@ledgerly/ds/tokens.css";
@import "@ledgerly/ds/tailwind.css";
```

```
<!-- template: normal Tailwind classes, now speaking Ledgerly -->
<button class="bg-primary hover:bg-primary-hover text-on-primary rounded-md px-md py-
sm">Add merchant</button>
<ds-stat-card label="Processed today" value="₹2.4 Cr"
trend="+8.2%" (dsSelect)="openReport($event)"></ds-stat-card>
```

Project A · Angular 20 + Tailwind 4 · production build, running



✓ Proof · Tailwind classes compiled to Ledgerly tokens

CLASS USED	WHAT TAILWIND GENERATED
`bg-primary`	`background-color: var(--ds-color-primary)`
`p-xl`	`padding: var(--ds-space-xl)`
`rounded-lg`	`border-radius: var(--ds-radius-lg)`
`shadow-card`	`--tw-shadow: var(--ds-shadow-card)`

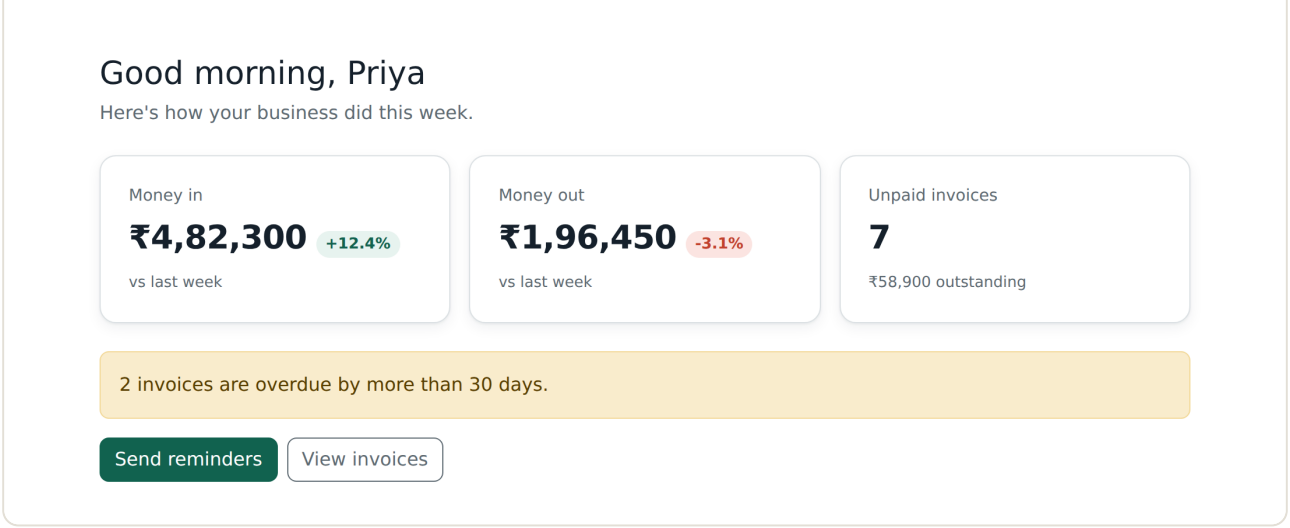
```
$ ng build
main.js     101.56 kB   styles.css   8.66 kB
Application bundle generation complete.
```

Project B · React + Bootstrap (Ledgerly Web)

```scss
// src/custom.scss — Ledgerly values first, then Bootstrap
@import "@ledgerly/ds/bootstrap.scss";
@import "bootstrap/scss/bootstrap";
```

```jsx
// App.jsx — ordinary Bootstrap classes plus the shared card
<StatCard label="Money in" value="₹4,82,300" trend="+12.4%" caption="vs last week" />
<div className="alert alert-warning">2 invoices are overdue…</div>
<button className="btn btn-primary">Send reminders</button>
```

Project B · React 19 + Bootstrap 5.3 · Vite production build, running



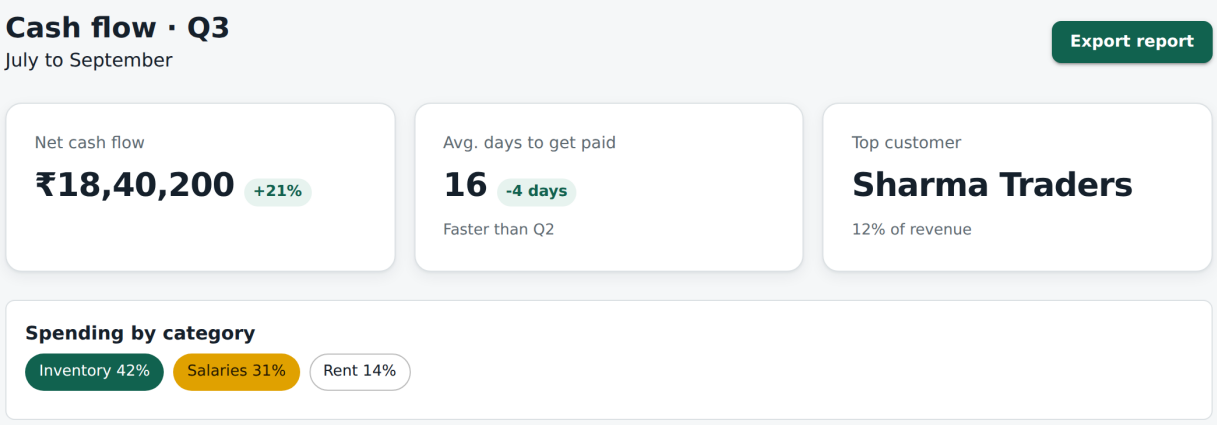
✓ Proof · Bootstrap repainted in Ledgerly's brand

```
$ grep in built CSS
--bs-primary: #11624f        ← was Bootstrap's default blue #0d6efd
--bs-btn-bg:  #11624f        ← every .btn-primary
ds-stat-card                 ← shared component bundled
```

Project C · React + Material UI (Ledgerly Insights)

```
// main.jsx — hand Ledgerly's theme to Material UI
import { ledgerlyMuiTheme } from '@ledgerly/ds/mui';
<ThemeProvider theme={createTheme(ledgerlyMuiTheme)}> … </ThemeProvider>
```

Project C · React 19 + Material UI · Vite production build, running



✓ Proof · Material UI uses Ledgerly's theme

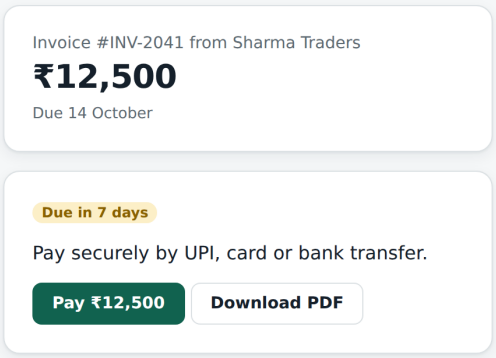
```
$ grep in built JS
#11624F      ← primary colour inside MUI theme
Manrope      ← brand font family
```

Standard MUI `Button`, `Card` and `Chip` components appear in Ledgerly green and saffron, with Ledgerly's corners and no uppercase button text.

Simple website · plain HTML, no framework

```
<link rel="stylesheet" href="tokens.css">
<link rel="stylesheet" href="styles.css">
<script src="components/ds-stat-card.js" defer></script>
<button class="ds-btn ds-btn--primary">Pay ₹12,500</button>
```

Invoice payment page · plain HTML + Ledgerly classes



Future · Vue + Vuetify

Code is included (`examples/future-vue-vuetify`) showing `createVuetify` with `ledgerlyVuetifyTheme` and the `LedgerlyDS` plugin. It was not built in this test, so treat it as a ready template rather than verified.

STEP 9, CONTINUED

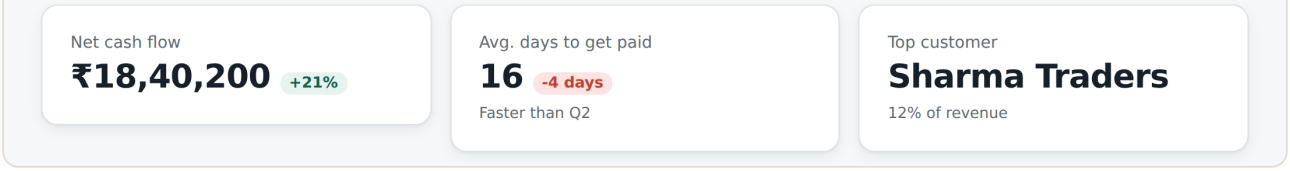
What the pilot caught, and how it was fixed

Exactly as the plan predicted, the first real use exposed problems. Each was fixed once, inside the design system, re-released as version 1.1.0, and every app inherited the fix without any app code being patched.

#	PROBLEM FOUND	CAUSE	FIX (IN THE DESIGN SYSTEM ONLY)
1	Angular app refused to build: "file is missing from the TypeScript compilation"	The Angular wrapper was shipped as raw source code	Added an Angular compile step (`ngc`, partial mode) to the package build, so Angular receives ready-made code
2	Cards with a caption were taller than their neighbours	The card didn't stretch to fill its grid cell	Card now fills the full height of its slot
3	"−4 days to get paid" shown in red	Colour was guessed only from the minus sign, but fewer days is good news	New `tone` input (positive / negative / neutral) overrides the guess
4	Plain Sass couldn't find `@ledgerly/ds/bootstrap.scss`	Plain Sass ignores package labels	No change needed: real projects use Vite or Angular CLI, which read the labels. Documented for the team

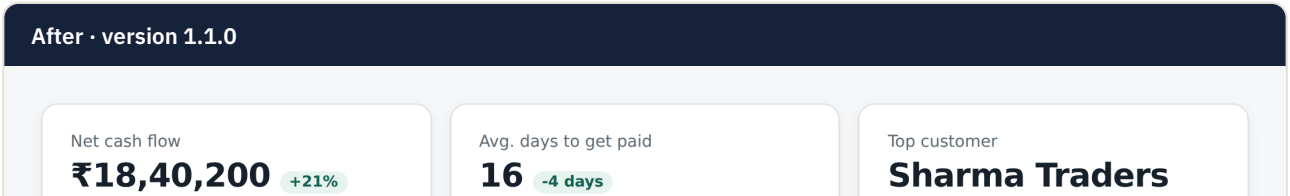
Before and after (fixes 2 and 3)

Before · version 1.0.0



Middle card taller than the first; "−4 days" wrongly red.

After · version 1.1.0



Equal heights; "−4 days" correctly green. Both captured from the same page, only the component version differs.

Step 10

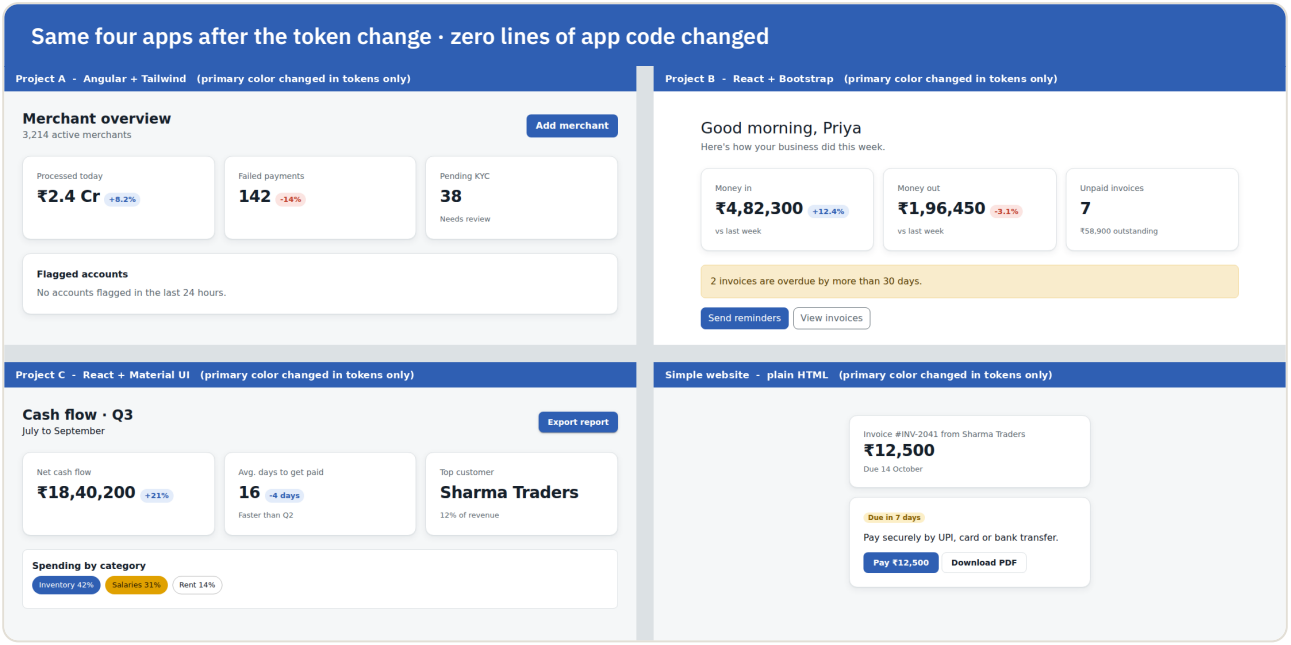
Proof that one change reaches every app

Owner: UX architect

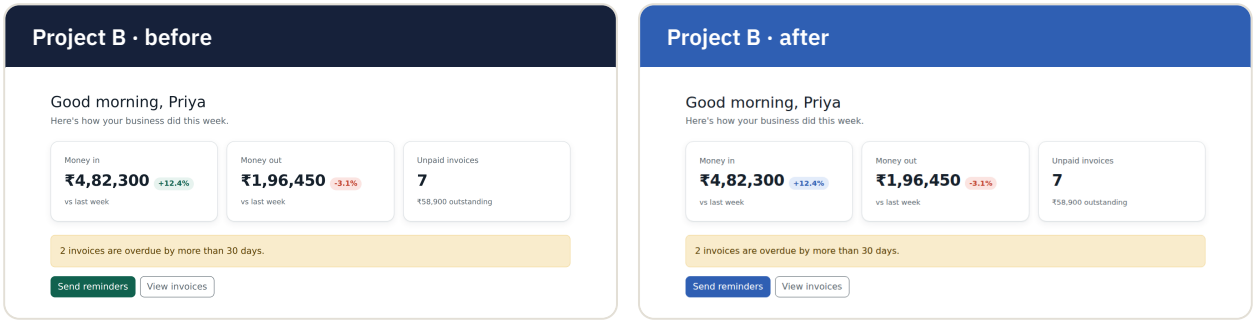
The final and most important test of the whole idea: change the brand colour in the token file only, rebuild, and check every app.

The only edit made

```
// tokens/semantic.json
"primary":        { "$value": "{palette.green.600}" }   ← before
"primary":        { "$value": "{palette.blue.600}" }    ← after
(primary-hover and primary-subtle moved to matching blues as well)
```



Buttons, Bootstrap's primary colour, Material UI's contained button and chip, Tailwind's `bg-primary`, and the trend chips all switched to blue. Warning, danger and neutral colours stayed put, because they point at different meanings.



The brand was then restored to green for the final package, to keep the delivered example consistent.

The change routine, now proven

Design Change in Figma → **Approve** UX architect edits tokens → **Build** `npm run build` → **Release** New version published → **Update** Apps install it

VERIFICATION

Everything that was tested

CLAIM	HOW IT WAS TESTED	RESULT
One token list generates all adapters	Ran Style Dictionary build; inspected every output file	**Pass · 6 files**
Meanings reference raw values	Checked `tokens.css` for `var(--ds-palette-…)`	**Pass**
Works as a real installed package	Packed tarball, installed into each app	**Pass**
Angular + Tailwind app builds	Angular CLI production build	**Pass (after fix #1)**
Tailwind classes use tokens	Inspected compiled CSS for 6 classes	**Pass**
React + Bootstrap app builds, repainted	Vite build; `--bs-primary` in output	**Pass · #11624f**
React + Material UI app builds, themed	Vite build; theme values in bundle	**Pass**
Apps run without errors	Opened in Chromium, listened for page errors	**Pass · 0 errors**
Pilot fixes reach all apps	Released 1.1.0, rebuilt all, re-captured	**Pass**
One token change re-brands everything	Changed primary to blue; rebuilt; captured all apps	**Pass**
Vue + Vuetify	Code written, not built	Not verified
Brand font loads	Font not bundled; fallback used	To do

HANDOVER

What's delivered, and what's next

In the zip (`ledgerly-ds-example.zip`)

FOLDER	CONTENTS
`tokens/`	The master list: 11 raw values, 14 meanings
`build/`	The translator script
`src/`	Plain classes, the web component, React/Angular/Vue wrappers
`dist/`	All generated compartments, ready to publish
`examples/`	Projects A, B, C, the plain page, and the Vue template
`README.md`	Team guide mapping every folder to the 10 steps

Recommended next steps for a production version

- [ ] Replace the example tokens with your own design system, exported from Figma via Tokens Studio.
- [ ] Load your brand font in each app, or add a `fonts.css` compartment that self-hosts it.
- [ ] Add Storybook as the shared preview gallery (Step 7).
- [ ] Move components to Stencil once there are more than a few, so wrappers are generated automatically.
- [ ] Add Changesets for version notes, and publish to a private npm organisation or GitHub Packages.
- [ ] Build and verify the Vue + Vuetify template when the first Vue project starts.
- [ ] Agree the golden rule with every team: no colours or spacing typed directly into app code.

> One master list, one translator, one package. Four apps with different toolkits, kept in sync by changing a single file.